Comparación de modelos y su costo por request (Bloque 9) / Lo que nadie ve del modelo: costo, latencia, escala

✓ **Hecho:** Ver

Lo que nadie ve del modelo: costo, latencia, escala

Hasta ahora trabajaste con un modelo a la vez, calculando el costo de una petición puntual.

> **Esta lección da un paso atrás**: en vez de mirar una sola petición, mira lo que pasa cuando esa petición se repite miles de veces, y por qué la elección del modelo detrás de tu aplicación no es un detalle técnico menor, sino una decisión con consecuencias reales en dinero, en tiempo de espera, y en qué tan bien tu aplicación se sostiene cuando crece.

No necesitas instalar nada nuevo ni ejecutar código para esta lección: es comparativa y conceptual. Si quieres probar por tu cuenta el ejemplo opcional de más abajo, alcanza con el mismo módulo que ya tienes funcionando desde el Bloque 4.

💰 Costo: ya sabes calcularlo, ahora se trata de comparar

Ya conoces la fórmula: **tokens** de entrada por su precio, más tokens de salida por el suyo. Lo que todavía no viste es que ese precio no es el mismo entre modelos, ni siquiera entre modelos del mismo proveedor.

> Un modelo más grande y más capaz casi siempre cobra más por token que uno más chico y más limitado, precisamente porque procesarlo cuesta más recursos del lado del proveedor.

Esa diferencia, que en una sola petición de prueba parece insignificante, deja de serlo apenas tu aplicación tiene uso real: la misma pregunta, respondida por un modelo el doble de caro, cuesta el doble multiplicado por cada usuario y cada interacción, todos los días.

Ejemplo

Comparar dos modelos para la misma aplicación

Imagina dos modelos candidatos para tu misma miniaplicación:

Modelo A

Cobra por token de entrada: **0.00000010 USD**
Cobra por token de salida: **0.00000030 USD/token**

Modelo B

Cobra por token de entrada: **0.00000050 USD/token**
Cobra por token de salida: **0.00000150 USD/token**

Con la misma petición de ejemplo que ya calculaste en la lección anterior

850 tokens de entrada + **320 tokens** de salida, así se ve la diferencia:

Modelo A: (850 × 0.00000010) + (320 × 0.00000030) = **0.000181 USD**
Modelo B: (850 × 0.00000050) + (320 × 0.00000150) = **0.000905 USD**

Diferencia por petición: 0.000724 USD

Aislada, esa diferencia parece irrelevante. Pero con **50.000 interacciones** al mes, un volumen modesto para una aplicación real, la diferencia entre modelos deja de ser un detalle: son **9,05 USD** con el Modelo A frente a **45,25 USD** con el Modelo B, todos los meses, por exactamente la misma funcionalidad.

Latencia: el tiempo que tarda, no solo lo que cuesta

Ya viste, en la lección de tokens, que el modelo genera la respuesta un token a la vez, y que por eso una respuesta larga tarda más que una corta.

Lo que falta agregar es que esa velocidad de generación no es igual entre modelos: en términos generales, un modelo más grande y más capaz suele tardar más en producir cada token que uno más chico, aunque procese la misma cantidad de texto.

> **A esa demora se le llama latencia**, y desde la perspectiva de quien usa tu aplicación, **es tan real como el costo**: una respuesta correcta que tarda ocho segundos en aparecer se siente peor que una respuesta ligeramente menos completa que aparece en dos.

En muchos productos reales, la latencia termina siendo tan determinante en la elección del modelo como la calidad de la respuesta.

A grandes rasgos, así se vería medir esto en código, si tienes curiosidad (opcional; el método completo, repitiendo la prueba varias veces y promediando, es el contenido de la siguiente lección):

```
import time

url, body = armar_peticion(pregunta)
inicio = time.perf_counter()
respuesta_json = enviar_peticion(url, body)
fin = time.perf_counter()

latencia = fin - inicio
print(f"Latencia: {latencia:.2f} segundos")
```

Con una petición real, algo como "Latencia: 1.84 segundos", aunque el número exacto varía en cada llamada. Precisamente por esa variación, la siguiente lección te pide repetir la medición varias veces por modelo y usar el promedio, no quedarte con un solo resultado.

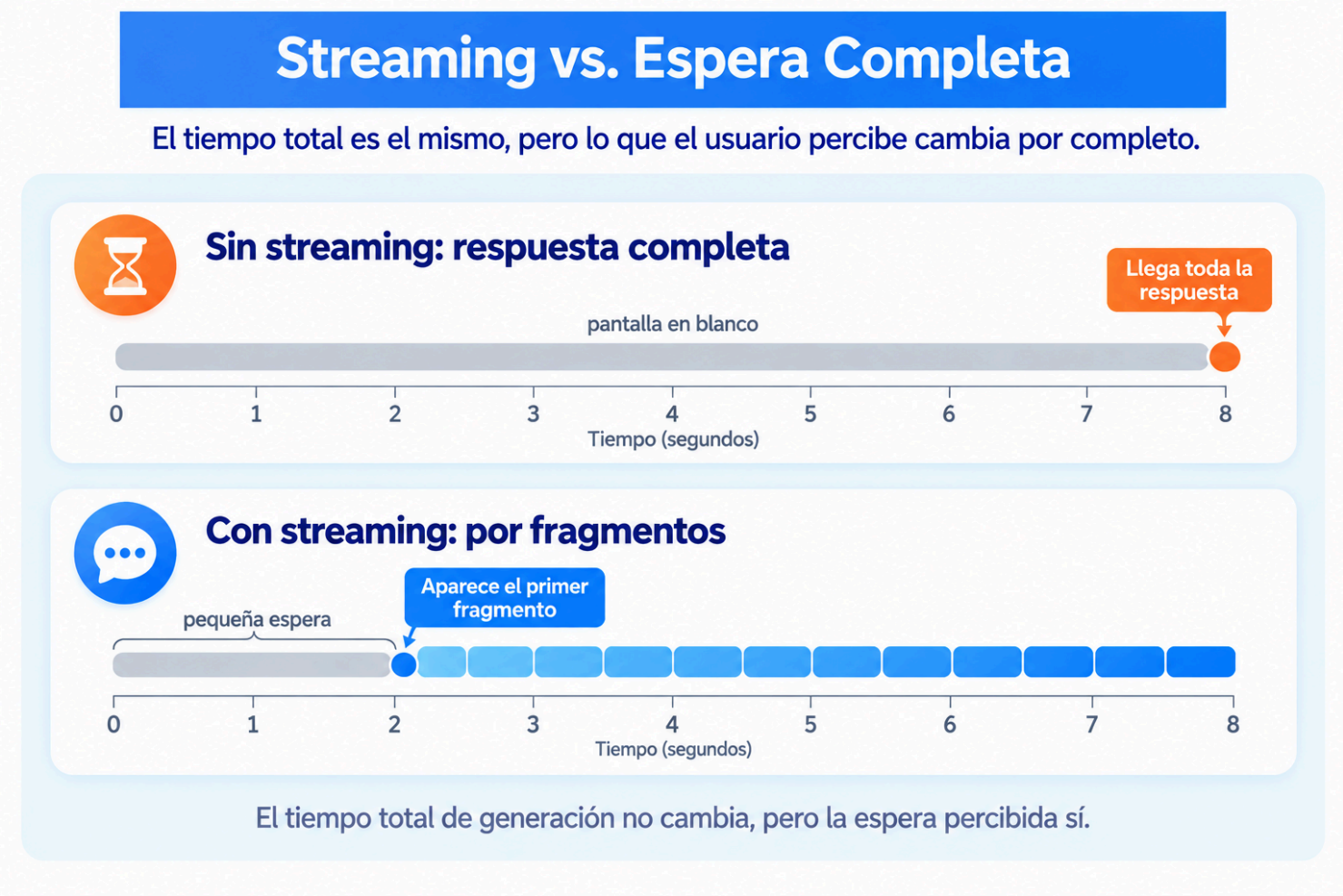


La latencia, además, tiene una segunda dimensión que vale la pena distinguir: cuánto tarda en aparecer el primer fragmento de la respuesta, no solo cuánto tarda en llegar completa.

Muchos **proveedores** permiten **"streaming"**: en vez de esperar a que el modelo termine de generar toda la respuesta para entregarla de una sola vez, el texto llega en fragmentos, a medida que se va generando, igual que cuando ves aparecer una respuesta palabra por palabra en un chat.

El tiempo total de generación no cambia con streaming, pero la latencia percibida por quien espera sí cambia, y bastante: ver que algo empieza a aparecer a los dos segundos se siente radicalmente distinto a una pantalla en blanco durante ocho segundos, aunque en los dos casos la respuesta completa demore exactamente lo mismo en terminar.

> No vas a implementar streaming en esta unidad: alcanza con que sepas que existe, porque es una de las primeras herramientas a las que se recurre cuando la latencia de una aplicación real se vuelve un problema, más adelante en el programa.



📈 Escala: lo que cambia cuando no es una petición, son miles

Una petición de prueba, la que hiciste tú para verificar que tu módulo funciona, no revela nada sobre cómo se comporta ese mismo modelo bajo uso real.

A escala, tres cosas que en una petición aislada pasan desapercibidas se vuelven centrales:

El costo se multiplica directamente por el volumen

Una diferencia de precio

La latencia se acumula si las peticiones dependen unas de otras

Una aplicación que hace

Los proveedores imponen límites de uso

Un límite de peticiones o de tokens por minuto,

por token que parecía insignificante se convierte

en una diferencia real en la factura mensual apenas hay cientos o miles de usuarios.

varias peticiones en cadena para responder

una sola pregunta del usuario multiplica la demora de cada una.

pensado para uso moderado, puede volverse

una restricción real cuando la aplicación crece, y es algo que conviene conocer antes de elegir un modelo, no después.

🚦 **Troubleshooting y escala**

Cuando superas el límite de uso, Gemini responde con el error HTTP **"429 RESOURCE_EXHAUSTED"**, no con un error de conexión: la petición llegó, pero el proveedor la rechazó por volumen. La propia documentación de Gemini recomienda esperar un momento y reintentar.

🔄 **Backoff exponencial:** si falla, esperar un poco antes del siguiente intento (por ejemplo 1 segundo), y si vuelve a fallar, esperar el doble (2, luego 4, luego 8…), en vez de reintentar inmediatamente una y otra vez, que solo empeora la saturación.

No hace falta que lo implementes ahora: alcanza con que sepas que existe y por qué, para cuando tu aplicación crezca lo suficiente como para necesitarlo.

🎯 No hace falta que tu miniaplicación de este programa maneje miles de usuarios reales para que esto importe: entender estos tres factores ahora es lo que te permite elegir con criterio en la próxima lección, en vez de elegir el modelo solo porque es el que ya venías usando.

⚖️ El triángulo costo, latencia y calidad

Estos tres factores (**costo**, **latencia**, y **calidad de respuesta**) no se pueden optimizar los tres al mismo tiempo: **mejorar uno casi siempre implica ceder algo en otro**.

Un modelo más grande generalmente da respuestas de mejor calidad, pero cuesta más y tarda más en generarlas. Pedir respuestas más cortas reduce el costo y la latencia, pero puede sacrificar detalle o precisión en la respuesta.

No existe una combinación que maximice los tres vértices a la vez. Lo que sí existe es una combinación

que se ajusta mejor a lo que tu caso específico necesita, y encontrarla, con datos concretos en vez de con intuición, es exactamente el ejercicio de la siguiente lección.



🎯 No existe "el mejor modelo", existe el mejor modelo para tu caso

> ⚠️ Es tentador buscar una respuesta única a "¿cuál es el mejor modelo?", pero esa pregunta está mal planteada: el **modelo más capaz no siempre es la mejor elección**, porque suele ser también el más caro y el más lento.

Para una tarea simple y repetitiva, un modelo más chico, más barato y más rápido puede dar un resultado igual de útil, a una fracción del costo.

La pregunta correcta no es cuál modelo es mejor en abstracto, sino **cuál modelo cumple mejor los requisitos concretos de tu caso**: la calidad de respuesta que tu aplicación realmente necesita, el costo que estás dispuesto a asumir por interacción, y la latencia que tus usuarios están dispuestos a tolerar.

> **Antes de seguir**: el ejemplo de 50.000 peticiones mensuales de esta lección es, en los hechos, lo mismo que 5.000 usuarios activos haciendo un promedio de 10 peticiones cada uno al mes. Pruébalo

con otro supuesto, usando la misma fórmula: si cada usuario hiciera el doble de peticiones (20 en vez de 10), ¿cuánto costaría el mes completo con cada modelo?

Siguiente lección

Bloque 9 · Cómo elegir modelos

Con estos tres criterios ya claros (**costo**, **latencia** y **escala**), en la siguiente lección los vas a aplicar directamente: vas a comparar dos modelos candidatos concretos para tu propia miniaplicación, y vas a usar esos criterios para argumentar, con datos y no solo con intuición, cuál conviene más.

Última modificación: jueves, 24 de septiembre de 2026, 16:26